Entitlement Status

The complete reference. Every value `entitlement_status` can hold, what writes it, how long each one takes to take effect, and how BlueHive learns that an Off-Grid Drive television has stopped being one — when the two backends have no network path between them and never will.

Scope
BlueHive (companion) · Off-Grid Drive (host) · bluehive-api · platform-api

Current as of
12 September 2026, after the unpair / machine-abandonment work

Supersedes
host-entitlement.pdf (8 September 2026), which predates `host_paired`

> **How to read this**
>
> **Part 1 is written for anybody** — no code, no jargon, no assumptions. If you only read one section, read that one. Parts 2 to 8 are the precise reference: schema, triggers, timings, SQL guards. Part 9 onwards is for operating the thing: what to check when a customer says they are locked out, and what is still imperfect. Every constant quoted is named, so you can grep for it rather than trust this page.

Part 1 Start here: the whole thing in plain English

What problem is being solved

Off-Grid Drive sells the subscription. BlueHive is one of the apps that subscription pays for. They are separate products with separate servers, and those servers are deliberately not connected to each other — there is no cable between them, no shared database, no API either can call. That is a security decision, and it is not going to change.

So when something changes on the Off-Grid Drive side — somebody stops paying, somebody unpairs their television, somebody uninstalls the app — BlueHive's server has no way to be told. The news has to travel some other way.

The news travels with the customer, in a sealed envelope

When Off-Grid Drive checks in with its own server, that server writes a short statement about the account — essentially *this household is paid up* or *this household is not* — and seals it with a signature only that server can produce. Off-Grid Drive keeps the sealed statement on the television and leaves it on a shelf where BlueHive can reach it.

BlueHive picks it up and carries it to its own server. That server inspects the seal, and because it holds the matching key it can tell a genuine statement from a forged one. It then replies with a single word describing the account, and BlueHive acts on that word.

The important part is what BlueHive is **not** doing. It never opens the envelope. It could not read the contents if it tried, and it certainly could not write one. It is a courier: it carries a sealed document between two offices and acts only on the reply. That is what makes the arrangement safe even though the envelope spends its whole life on a device the customer owns and could tamper with.

The shelf answers two questions, not one

For a long time the shelf held one thing: the envelope. That turned out to be ambiguous in the worst possible way. An empty shelf could mean *I have been unpaired and have nothing for you*, or it could mean *my app only just woke up and has not fetched anything yet*. Those want opposite responses, and BlueHive could not tell them apart, so it had to assume the harmless one. That is exactly how an unpaired television kept working.

The shelf now has a second slot, and it holds a plain yes-or-no: **does this television still have an Off-Grid Drive account signed in on it?** Off-Grid Drive answers that from its own records rather than by looking at the envelope, so it is still correct in the moment after pairing when there is an account but no envelope yet. That one extra slot is what makes unpairing enforceable.

Three different ways a television can stop being entitled

These get confused constantly, so they are worth separating once and for all. They are genuinely different facts, they are recorded as different values, and they clear in different ways.

What happened	What it is a fact about	Recorded as	How it comes back
Payment stopped	The **account**. Nothing is wrong with the television.	**inactive**	Subscribe again. Every television on the account recovers.
Unpaired, or Off-Grid Drive uninstalled	This one **television**. Says nothing about the subscription or about the household's other sets.	**abandoned**	Pair that television again. Only that one was affected.
Switched off by hand	The **account**, by deliberate operator action.	**blocked**	Only a human can undo it. Nothing automatic ever will.

The three ways out, and what each one actually claims.

What the customer sees

One screen, and it is deliberately vague. It says *"Connection trouble — BlueHive is currently having trouble connecting. This is usually temporary, try again in a moment."* with a **Retry connection** button. It does not say *your subscription lapsed* or *this device is not paired*, because the app genuinely cannot always tell the difference between a refusal and a bad night on the network, and guessing wrong in public is worse than saying little.

The Retry button is not decoration. Pressing it goes and asks Off-Grid Drive again — and the act of asking is what wakes Off-Grid Drive up if Android had shut it down, which is what makes it fetch a fresh statement. Somebody who has just re-subscribed or just re-paired gets back in on the first press, without reinstalling anything.

The rule that matters more than any other: it fails open

Exactly one reply locks the app: the word `inactive`. Everything else lets the customer straight through — `active`, `unknown`, a word a future server invented, an empty reply, a request that timed out, no Off-Grid Drive on the television at all.

That lopsidedness is the single most important decision in the design, for two reasons. First, BlueHive can be installed on its own from GitHub, with no host app anywhere near it; those installs are free and always have been, and locking on anything short of a definite refusal would shut out the entire organic install base. Second, *I could not reach the server* and *the server says no* arrive looking almost identical from inside an app. Only one of them is evidence.

> **Put plainly**
>
> The system is built to occasionally let in somebody who should not be there, rather than ever lock out somebody who should be. The first mistake costs a little revenue. The second costs a customer, a support conversation, and a good deal of trust.

And the lock does not depend on the app behaving itself

None of this rests on BlueHive being honest. Even if the app were modified to ignore its own lock screen entirely, BlueHive's **server** checks the same fact independently on every request and simply refuses to serve a television that is not entitled. The copy held on the device exists for speed and for showing a sensible screen — not for enforcement. The

device is only the courier's last leg.

Part 2 The column itself

One column carries all of this. It lives on the device table, not the user table, and that relocation is load-bearing rather than housekeeping.

```
ALTER TABLE bluehive.devices
ADD COLUMN IF NOT EXISTS entitlement_status VARCHAR(20) NOT NULL DEFAULT 'unknown';

ALTER TABLE bluehive.devices
ADD CONSTRAINT ck_bh_devices_entitlement_status
CHECK (entitlement_status IN ('unknown','active','inactive','abandoned','blocked'));
```

accounts.py, lines 155 and 184. Modelled in *bh_models.py* line 413. The `CHECK` constraint is the authoritative list — five values, no others, and the database will refuse a sixth until somebody adds it here deliberately.

Why it is per-device and not per-account

It used to live on `bluehive.users` and was dropped from there on 22 August 2026. A single per-account value cannot express the thing the product needs to express: that one of a household's four televisions lost its host while the other three are perfectly fine, and the subscription paying for all four is untouched. Abandonment is a fact about hardware. It needs somewhere per-hardware to live.

Account-wide reach still exists where it belongs — a lapsed subscription and an operator block both reach every device on the account — but it is produced by the *query* in Part 6, not by the column's location. See Part 8 for exactly where each boundary falls.

The five values at a glance

Value	What it asserts	Blocks ?	Written by	Cleared by
unknown	No opinion. Either nothing has ever reported, or a report arrived that could not be mapped onto a known standing.	**no**	The default on every new device row. Also written deliberately when a live host clears an abandonment.	n/a — this is the resting state.
active	A valid, unexpired, correctly signed pass said the account is in good standing.	**no**	`host_entitlement.py`, on a pass carrying `st=active`.	Any later report; the hourly expiry sweep.
inactive	The subscription is not in good standing, or the pass proving it has expired.	**YES**	A pass carrying `st=lapsed`; any expired pass; the hourly expiry sweep.	A fresher pass saying `active`.
abandoned	This **machine** no longer has a host on it — unpaired, or Off-Grid Drive uninstalled. Says nothing about the subscription.	**YES**	`host_present=false` or `host_paired=false` on a report, and only on a device that has shown a real pass before.	A pass minted *strictly later* than the mark. Re-pairing produces one.
blocked	An operator switched this account off by hand. The kill switch.	**YES**	Nothing in the codebase. Manual `UPDATE` only.	A human, manually. Pinned against every automatic writer.

Three of the five block. `ENTITLEMENT_BLOCKED = frozenset({'inactive','abandoned','blocked'})` — bh_auth.py:153.

> **Two of the five never reach the client by name**
>
> `abandoned` and `blocked` are both reported to the app as the single word `inactive`. The app only needs to know whether it is refused, and collapsing them means a curious person reading BlueHive's network traffic cannot tell “your subscription lapsed” from “we marked this box” from “an operator switched you off”. The distinction is kept in the database, where it is useful, and dropped on the wire, where it would only be intelligence. *host_entitlement.py*, lines 495–523.

Part 3 The five values in detail

unknown — the honest default

Every device row starts here, and a great many stay here forever. A BlueHive installed from GitHub with no host app never reports anything at all (Part 9 explains why the report is skipped rather than sent), so its row sits on `unknown` for life and the app works normally.

It is also written *deliberately* in one place: when a live host clears an abandonment across a machine, sibling rows belonging to other accounts are reset to `unknown` rather than to `active`. Those accounts have not proved anything; they are returned to “we do not know yet” and re-establish their own real standing on their next report.

active — a valid pass said yes

Written when a correctly signed, unexpired pass carries `st=active`. On the issuing side that word comes from exactly one function, `_is_entitled()` in *events.py*, and since 23 August 2026 only the Stripe status `subscribed` passes it. `past_due` — a charge Stripe is still retrying — used to pass, which meant a failed card cost nothing for the whole retry window, typically weeks.

There is one function rather than two because there are two consumers — the pass mint that BlueHive reads, and `/api/android/entitlement` that Off-Grid Drive reads — and while each carried its own copy of the rule, narrowing one produced the worst possible state: Off-Grid Drive wide open on a failed card while its own add-on sat behind a lock screen. A customer cannot make sense of that, and neither could support.

inactive — the subscription is not in good standing

Three separate paths land here, and they are worth knowing apart because only the first is fast:

- **A pass that says so.** The host mints `st=lapsed`; the verifier translates that to `inactive` on arrival. The rename is deliberate — the claim name is a contract between two separately deployed products, so renaming it at the source would need a coordinated deploy and would misread every token already in the field.
- **An expired pass, whatever it says inside.** Once past its expiry the stated standing is ignored entirely. This is what stops the obvious cheat of hoarding an old `st=active` pass and replaying it forever.
- **The hourly expiry sweep.** A device whose pass ran out and which has simply stopped reporting gets flipped server-side, without anybody asking. Details in Part 5.

This is the only value the client treats as a lock. `STANDING_INACTIVE = "inactive"`, matched exactly and lowercased — *HostEntitlementRelay.kt:103*.

abandoned — this machine no longer has a host

The newest of the five and the subtlest. It says *there is no host on this television any more*, which is a fact about the **device** and outlives whichever account happens to be signed in on it.

Why it cannot just be folded into `inactive`. It is what closes the account-hopping bypass: strip Off-Grid Drive off a television, make a brand-new BlueHive account, and that new account has no history of its own, so nothing else in the system has any reason to refuse it. The server checks whether the *machine* was abandoned, matched on the device fingerprint — Android's own per-device identifier, which survives uninstalling and reinstalling BlueHive and survives

clearing its data, and changes only on a factory reset. A new account on stripped hardware therefore inherits the mark. Fold this into `inactive` and that inheritance rule loses the only thing it can key on.

Unpairing counts the same as uninstalling, deliberately. Both mean the same thing to a viewer: this television no longer has a host behind it. The product rule is that BlueHive obtained *through* the host stays available only while the host does, so the two ways of removing it land on the same standing rather than one being enforced and the other waited out.

It can only ever be written to a television that once showed a real pass. The `WHERE` clause requires `host_seen_at IS NOT NULL`, so the mark needs a validly ES256-signed assertion in the device's history. That is proof, not a claim — unforgeable without a private key this backend does not hold. A device that never met a host cannot be abandoned by asserting that it was.

> **Reversibility is not optional here**
>
> Somebody who uninstalls Off-Grid Drive for ten minutes to free up space has to be able to get back. Re-pairing mints a fresh pass, and the guard on the write lifts the mark for any pass minted *strictly later* than the abandonment. Replaying the stale mirrored copy cannot lift it — that asymmetry is the entire reason this is not a bypass. It is also why `abandoned` is the right value rather than something harsher and permanent.

`blocked` — the manual kill switch

Nothing in the codebase writes this value. It exists to be set by hand, by an operator, against a real situation. Its defining property is that it is **pinned**: every automatic writer in the system is explicitly arranged not to clear it.

```
SET entitlement_status = CASE WHEN entitlement_status = 'blocked'
THEN 'blocked' ELSE :ent END,
```

Without that `CASE` the switch would look broken rather than absent: you set it by hand, the television launches a minute later, and the ordinary report writes `active` straight over the top. The operator sees an account they just switched off still working, with no way to tell that anything happened.

Every *other* column is still written while blocked — `host_seen_at`, the pass, the mint time — so a blocked account stays visible in the health census and you can still see whether that television is being used. Only the verdict is held.

It is also the one value that is **account-wide by design**. A kill switch that only covered the television it was typed against would be escaped by signing in on anything else, which is the opposite of a kill switch. Set it on any one of an account's devices and the account is refused everywhere.

```
-- block an account (every device, immediately)
UPDATE bluehive.devices SET entitlement_status = 'blocked' WHERE user_id = :uid;

-- release it; 'unknown' lets the next report establish the truth
UPDATE bluehive.devices SET entitlement_status = 'unknown'
WHERE user_id = :uid AND entitlement_status = 'blocked';
```

Part 4 How the data crosses: two backends that never speak

This is the part people find hard to believe, so it is worth laying out end to end. There is no connection between platform-api (Off-Grid Drive's backend) and bluehive-api. Not a firewalled one, not an internal one — none. The television is the only thing that touches both.

#	What happens	Where	Code
1	Off-Grid Drive asks its own backend for a statement of account standing. The backend signs it with ES256 using a private key it alone holds.	OGD → platform-api	`GET /api/android/entitlement-assertion`
2	Off-Grid Drive caches that pass on the television and publishes it read-only, alongside a plain yes/no for whether it still holds an account.	on the TV	`HostAssertionProvider`
3	BlueHive reads both. Three sources for the pass, in priority order: an extra handed over at launch, then the host's shelf, then its own mirrored copy.	on the TV	`HostEntitlementRelay.submit`
4	BlueHive posts it to its own backend, which verifies the signature with the **public** half of the key and writes a verdict to the device row.	BlueHive → bluehive-api	`POST /api/bluehive/host-entitlement`
5	The server replies with one word. Only the exact string `inactive` locks the app; everything else is let through.	on the TV	`applyStanding`
6	A lock is announced on an in-process bus and the lock screen is raised over whatever was on screen.	on the TV	`EntitlementLockBus` → `ServiceLockoutActivity`

The whole chain. Steps 1 and 4 are the only network calls, and they are to different servers that know nothing of each other.

What is actually inside the envelope

Claim	Meaning	Notes
`sub`	Who the account is	An HMAC that only the issuer can reverse. Not an email, not a user id, not a device id — BlueHive learns nothing identifying from it.
`lic`	Which licence group	Lets the stream cap apply across an Off Grid Labs licence. Absent on older passes, treated as *not known* rather than *no licence*.
`st`	The standing itself	Exactly two values ever emitted: `active` or `lapsed`. Anything else is recorded as `unknown` rather than guessed at.
`iat`	When it was minted	**The ordering key for every write.** See Part 7 — this replaced `exp` after a real outage.
`exp`	When it stops counting	`iat` + `HOST_ASSERTION_TTL_DAYS`, which is **2 days** (env-overridable, *events.py:378*).
`iss`	Issuer label	The literal string `host`.

Signed ES256. BlueHive's backend holds only the public half, so it can verify and cannot mint.

> **The one failure mode that is invisible, and how it is caught**
>
> If the keypair ever drifts apart — a rotation that updates one side, a bad paste, a mangled multi-line environment variable — then every pass is rejected with a 400 and `entitlement_status` silently stops changing. On a small user base that is indistinguishable from “nobody used the companion this week”. Because the two backends must never talk, neither can ask the other whether the halves still match. So each derives the *same* 16-hex fingerprint from the half it holds and logs it at boot, and an operator compares two log lines. Nothing secret is transmitted or logged — the fingerprint is of the public key, which the verifier already holds by design. **Compare those two lines after any key rotation.**

The shelf, precisely

```
content://io.host.entitlement.provider/assertion

assertion TEXT the signed pass, or NULL
paired INTEGER 1 = this host holds a session, 0 = it does not, NULL = did not check
```

The authority names a capability, not a product. `io.host.entitlement.provider` is vendor-neutral on purpose, so a companion declaring it in `<queries>` is not visibly tied to Off-Grid Drive, and any host implementing the same convention satisfies it. No Off-Grid Drive package name is hardcoded in BlueHive anywhere.

Always exactly one row now, where an absent pass used to produce zero rows. The row is the carrier for `paired`, and that fact has to survive the case where there is no pass — which is precisely the unpaired case it exists to report. Emitting nothing would throw the answer away exactly when it *is* the answer. Older companions built against the single-column shape are unaffected: they read the pass by name and null-check it, so a row with a null pass behaves for them exactly as zero rows did.

It is cache-only, never a network fetch. A provider query blocks the calling thread, and the caller here is another app's cold start.

> **Why pulling, and why pushing was never enough**
>
> The pass used to travel only as an Intent extra when Off-Grid Drive launched an add-on. That path dies exactly when it matters most: the moment an account lapses, Off-Grid Drive shows its own lock screen, so nobody can reach the Add-ons list, so nothing is ever launched from there again — and the companion is left holding whatever it last heard, which is `active`. The one fact worth delivering is the one the push model can never deliver. A provider inverts it: the companion opens from its own launcher tile, reads the shelf, and gets the current answer whether or not Off-Grid Drive is usable.

Part 5 Timings: what fires when, and how long each takes

Three independent clocks are involved and they stack. Off-Grid Drive decides how often a fresh pass is minted; BlueHive decides how often it carries one; and the server sweeps independently for passes that simply ran out. Nothing here is a fixed poll — almost every trigger is an event.

Clock 1 — when Off-Grid Drive mints a fresh pass

Trigger	Fires when	Rate limit	Effect
App start	Off-Grid Drive's main activity opens.	`REFRESH_AFTER_MS = 6h`	Mints only if the cached pass is absent or older than 6 hours.
Event stream snapshot	The server pushes account state on every stream connect, including reconnects.	deduped on last seen status	Forces a mint when the status differs from what this process last saw — and after a process start that is always, so a restart always resyncs.
Subscription changed	A subscribe or unsubscribe arrives on the stream.	same dedupe	Forces a mint immediately, in both directions. A re-subscribe that left a stale `lapsed` pass in the wallet is the same bug inverted.
Hourly top-up tick	`ASSERTION_TOPUP_TICK_MS = 1h`	hands off to the 6h gate	**The backstop that makes the whole thing safe.** Guarantees something asks even for a paying, correctly paired household that never opens Off-Grid Drive at all. Ticks hourly, mints roughly every 6 hours.

Trigger	Fires when	Rate limit	Effect
Pairing completes	A television finishes pairing.	none — forced	Mints immediately. **This is what makes an abandoned machine recoverable** — see Part 7.
Host lock screen retry	Somebody presses retry on Off-Grid Drive's own lock screen.	none — forced	Mints immediately, so a fresh subscription is reflected on the first press.
Unpair	Unpaired from the website, or from Off-Grid Drive's settings.	n/a	**Clears the cache entirely** and flips `paired` to 0. No further mints happen at all.

Five paths force a mint, two are gated at 6 hours, and the hourly tick exists purely so that a television left switched on for days cannot let its pass expire underneath a paying customer.

> **Why the hourly tick has to exist**
>
> Every other mint path is edge-triggered, and between them they miss the commonest case entirely. App start needs somebody to open Off-Grid Drive. The lock screen only runs once an account has already lapsed. The stream branch dedupes on purpose, so a stable subscription reconnecting all day never re-mints. That left exactly one thing refreshing the pass for somebody who is paying, correctly paired, and simply never opens Off-Grid Drive: the process being restarted by Android — which on a television left switched on can be days apart, comfortably longer than the 2-day pass. The pass would expire while paired and paying, and BlueHive would lock its user out on the strength of it.

Clock 2 — when BlueHive carries a pass to its own server

Trigger	Fires when	Floor	Sees unpair?	Sees uninstall?
Launch	Every BlueHive cold start, from `AuthActivity`.	none	**yes**	**YES**
Launch, second pass	6 seconds after launch, and skipped entirely if nothing changed.	`SECOND_PASS_DELAY_MS = 6s`	**yes**	**no**
Home screen resume	Every time the user lands on the home screen — which is every route back into the app.	`CHECK_FLOOR_MS = 3s`	**yes**	**no**
Lock screen retry	Somebody presses **Retry connection**. Waits 4s and re-reads if the first answer was a refusal.	`RECHECK_WAKE_MS = 4s`	**yes**	**no**
Server 403	Any gated API call is refused. Locks locally with no further asking.	none	**n/a**	**n/a**

The uninstall column is the important one, and it is not an oversight — see directly below.

Why uninstall is caught on launch and only on launch

When Off-Grid Drive is gone, there is no provider to query. The home-screen hook reads the shelf, gets nothing — not a `false`, just nothing — and returns without reporting, because an empty read is genuinely ambiguous mid-session. The launch path behaves differently: it falls back to BlueHive's own mirrored copy of the pass, so it always has something to post, and it attaches `host_present=false` alongside it. Android's package manager is asked directly, which is a different and unambiguous question from *did the shelf have anything on it*.

Unpairing is the opposite shape: the package is still installed and perfectly healthy, so the shelf is still there and still answers — it just says `paired = 0`. That is an explicit claim rather than an absence, which is why every trigger can act on it.

> **Changed on 12 September 2026**
>
> Host absence used to require three separate readings spanning ten minutes before BlueHive would report it, and readings only accrued on a cold start — so uninstalling Off-Grid Drive meant opening BlueHive three times over ten minutes before anything happened. That debounce is gone. The rule is now simply: **no host, no access, decided the moment we look**. The debounce existed because Android reports a package as missing while it is being updated, so a badly timed launch can now mark a healthy machine — but the mark is reversible and a returning host clears it, so a permanent ten-minute hole was traded for a self-healing window a few seconds wide. That is the right way round.

Clock 3 — what the server does on its own

Job	Schedule	What it changes
Expiry sweep	Every hour at **:30**, plus once at container start.	Flips `active` or `unknown` to `inactive` on any device whose pass has expired. **An allowlist, not a denylist** — see below.
Health census	Daily at **05:30**, plus once at container start.	Reports only, changes nothing. Counts how many devices sit in each standing and warns if reporting has gone quiet for more than `ENTITLEMENT_STALE_AFTER_DAYS` = 2 days.

`ENTITLEMENT_HEALTH_HOUR` = 5, `ENTITLEMENT_HEALTH_MINUTE` = 30. Both run in the *bluehive-tasks* container, under `RUN_BACKGROUND_TASKS=1` — not in the API container.

```
UPDATE bluehive.devices
SET entitlement_status = 'inactive'
WHERE host_seen_at IS NOT NULL
AND host_expires_at IS NOT NULL
AND host_expires_at < now()
AND entitlement_status IN ('active', 'unknown') -- ALLOWLIST
```

That last line names the two standings that are *allowed* to decay, and it is the whole point of the clause. Written as a denylist it was correct on the day it was written and silently wrong the moment `abandoned` was added: an abandoned device's pass is expired *by definition*, because the host is gone and nothing is left to renew it, so this sweep would have found every abandoned row on its very next tick and flattened it to `inactive` — quietly disarming the account-hopping defence, with nothing appearing broken, until somebody made a fresh account on a stripped television and walked straight in. A denylist fails open on every status added after it; an allowlist fails closed.

The sweep is **hourly and the cadence is load-bearing**. It is what actually applies expiry, so its period stacks on top of the 2-day pass as pure additional lag. Nightly would mean a host that vanished keeps vouching for up to three days — half again as long as intended, for no reason but the schedule. An hour is noise next to two days.

Putting the three clocks together: end-to-end latency

What the customer did	How BlueHive finds out	Typical	Hard ceiling
Stopped paying	Off-Grid Drive is told on its event stream and force-mints a `lapsed` pass within seconds. BlueHive carries it on its next launch or home-screen resume.	seconds to the next screen change	2 days (pass TTL) + 1 hour (sweep), after which the server refuses outright and the app locks on the 403.

What the customer did	How BlueHive finds out	Typical	Hard ceiling
Unpaired the television	The shelf flips to `paired = 0` at once. BlueHive acts on it at its next launch *or* any home-screen resume.	immediate at next launch; seconds if the app is open and the user navigates	2 days + 1 hour. The cache was wiped, so nothing re-mints and the mirrored pass simply runs out.
Uninstalled Off-Grid Drive	Android's package manager is asked directly on the **launch** path.	the very next BlueHive launch	same 2 days + 1 hour if BlueHive is never launched again.
Re-paired the television	Pairing force-mints a pass with a brand-new mint time, which is what the un-abandon guard needs.	next launch, or one press of **Retry connection** (about 4s)	n/a — nothing needs to expire for recovery.
Re-subscribed	Force-mint on the stream event, then the same carry.	one press of **Retry connection**	n/a
Operator set blocked	The server refuses on the next gated request.	the next API call the app makes	n/a — this one does not wait for the device at all.

The ceilings all share the same shape: if the device never reports again, the pass it is holding expires and the hourly sweep finishes the job. Nothing depends on a device cooperating.

> **One honest gap**
>
> An unpair is not pushed to BlueHive. Off-Grid Drive does broadcast identity loss to registered companions over IPC, but **BlueHive is not registered as one** — it deliberately knows nothing about Off-Grid Drive's interfaces, which is what keeps it sideloadable. So a BlueHive left sitting on its home screen, untouched, with no navigation and no API calls, will not be kicked out the instant you unpair; it finds out on its next launch or its next screen change. Every route back into the app passes through the home screen, so in practice that is seconds. Closing the gap entirely would mean registering BlueHive as a companion and naming Off-Grid Drive in its manifest.

Part 6 The four refusal rules on the server

The column is evidence. The decision is made by one query in `bh_auth.require_user`, which runs on every gated request and asks four questions at once. Any one of them refusing produces a `403 entitlement_inactive`.

#	Rule	Question it asks	Scope	Matches on
1	**dev_status**	What does *this* television's own standing say?	this device	device id + user id
2	**machine_gone**	Has this physical machine been abandoned by *any* account that ever signed in on it?	this machine, every account	`device_fingerprint`
3	**account_lock**	Does this user have an *established* host that has gone bad?	the whole account	user id, plus a span test
4	**account_blocked**	Has an operator switched this account off by hand?	the whole account	user id

All four fail open. A missing row, a NULL, or a database error yields no refusal — a query returning nothing is not evidence of a bad standing.

Rule 3 is the one with the interesting history

An account-wide rule is necessary: a subscription is a property of the account, so a spare television that never met the host should not outlive it. But locking purely per-account meant one evening signed in at a friend's house locked you out at home, permanently, for nothing.

The span test separates those. Only a host you have been reporting for `HOST_BINDING_DAYS` — **7 days**, `BH_HOST_BINDING_DAYS` — gets to speak for your whole account. A visitor never accumulates a span, so a visit can never lock your home.

```
EXISTS (SELECT 1 FROM bluehive.devices e
WHERE e.user_id = :uid
AND e.host_seen_at IS NOT NULL
AND e.host_first_seen_at IS NOT NULL
AND e.host_seen_at - e.host_first_seen_at >= interval '7 days'
AND e.entitlement_status = 'inactive') -- 'abandoned' removed 2026-09-12
```

> **The change that made unpair safe**
>
> That last line used to read `IN ('inactive','abandoned')`. With `abandoned` in an **account-wide** rule, unpairing one of four paired televisions locked BlueHive on *all four*, and on every member account under the licence — precisely the collateral damage this rule's own comment says it was designed to avoid. Nothing is lost by removing it: an abandoned machine is still refused twice over, by rule 1 for that device and by rule 2 for that hardware under any account. Uninstall the host from every television and every television locks, one at a time, which is the correct shape.

Part 7 The write guards that keep the column honest

Devices report out of order, replay old passes, and come back from the dead. Six guards on one `UPDATE` are what make the column trustworthy anyway.

Guard	The clause	What it prevents
Order on mint time, not expiry	`host_minted_at IS NULL OR host_minted_at <= :minted`	An older report overwriting a newer one. It compared `exp` originally, on the reasoning that expiry is mint time plus a fixed TTL and therefore always rising — which held right up until the TTL itself changed. Shortening it from 7 days to 2 meant every device still holding a 7-day pass had a stored expiry further in the future than any 2-day pass could offer, so the guard rejected everything. Reporting froze dead for a week and logged only the reassuring `[SUPERSEDED]`.
Clamp mint time to now	`minted = min(minted, now())`	A clock jump on the issuer wedging the device permanently. A mint time in the future makes every correctly-timed mint that follows look older and get rejected. Clamping loses nothing: a pass whose mint time is ahead of our clock was minted, at the latest, now.
Abandonment needs history	`:ent <> 'abandoned' OR host_seen_at IS NOT NULL`	A device that never met a host being marked abandoned by simply claiming it was. The mark requires a validly signed pass in the device's past.
Leaving abandoned takes a *strictly newer* pass	`entitlement_status <> 'abandoned' OR :ent = 'abandoned' OR host_minted_at IS NULL OR host_minted_at < :minted`	**The rule that makes this not a bypass.** The main guard is `<=`, so re-posting the same pass keeps `host_seen_at` moving and proves the television is alive. Without this extra clause that same harmless re-post would flip an abandoned machine back to `active` — delete the host, wait for the mark, re-post the mirror you kept, and you are back in.

Guard	The clause	What it prevents
blocked is pinned	`CASE WHEN entitlement_status = 'blocked' THEN 'blocked' ELSE :ent END`	The manual kill switch being silently overwritten by the next ordinary report.
Licence claim is merged, not replaced	`COALESCE(CAST(:lic AS VARCHAR), host_license)`	An older host build, or one mid-rollout, blanking a licence we already know on every report — which would drop the device back to per-account stream counting while looking like it still worked.

Recovery has to clear what the check reads

When a live host reports `active` and the write lands, a second statement clears the abandonment mark from the *whole machine*, not just the reporting row:

```
UPDATE bluehive.devices SET entitlement_status = 'unknown'
WHERE device_fingerprint = (SELECT device_fingerprint
FROM bluehive.devices WHERE id = :did)
AND entitlement_status = 'abandoned'
AND (host_minted_at IS NULL OR host_minted_at < :minted)
```

The mark is matched on the device fingerprint across every account that has ever signed in on that television, so clearing only the one row would leave a stale `abandoned` on some other account's row — and *that* row would keep the machine locked for everybody, including the person who just reinstalled the host to fix it.

Note the two gates. **Only on** `active`, because an expired or lapsed pass proves nothing about whether the host is back. And **the same strictly-newer mint comparison**, because without it this is a second door to the same bug: another account signed in on the same television, holding its own stale mirror, could clear an abandonment it has no fresher evidence against.

And the endpoint answers with the row, not with its own arithmetic

The final verdict is read back out of the database after the write and returned from there. The two outputs of this endpoint had drifted apart once: what gets *written* is guarded by all of the above, while what got *returned* was guarded by none — so the server could decline to record a verdict and tell the client to lock on it in the same breath. The client obeys, because that is the whole design, so it raised its lock screen over a database that considered the account perfectly fine, and no 403 ever came to back it up. Retry could not clear it either, because retrying recomputed the same answer. Only clearing app data escaped.

Part 8 Scope: what unpairing one television does, and does not do

This is the requirement that shaped most of the design, so here it is as a worked example. A household has four televisions paired to one Off Grid Labs licence — an admin account with two, and a member account with two. The admin unpairs television #1.

	Off-Grid Drive	BlueHive	Why
TV #1 (unpaired)	**kicked to pairing screen**	**locked**	Rules 1 and 2 both refuse it: its own row is `abandoned`, and the machine itself is marked.
TV #2 (same account)	**unaffected**	**unaffected**	Different fingerprint, own row untouched, and `abandoned` is no longer in the account-wide rule.

	Off-Grid Drive	BlueHive	Why
TV #3 and #4 (member account)	**unaffected**	**unaffected**	Different account entirely; nothing crosses the licence.
A new BlueHive account, signed in on TV #1	n/a	**still locked**	**The account-hopping defence.** Rule 2 matches the hardware fingerprint, not the account, so a fresh account on a stripped television inherits the mark.
TV #1, after re-pairing	**works**	**works**	The forced mint at pairing produces a strictly newer pass, which lifts the mark on that row and on every sibling row for that machine.

The blast radius of an unpair is exactly one television, and it is reversible on that television alone.

> **The product rule this implements**
>
> Install BlueHive straight from GitHub and that is your business — the app is free and stays free. But a BlueHive obtained *through* Off-Grid Drive is part of what the subscription pays for, so uninstalling or unpairing the host costs you access to it, and you see the generic connection screen. That consequence lands on the specific television you unpaired, and on nothing else: not your other sets, and not the member accounts on your licence.

Part 9 The sideload rule, and why it genuinely holds

A reasonable worry about the change on 12 September 2026 is this: if BlueHive now reports `host_present=false` the instant it finds no host, does a plain GitHub install not mark *itself* abandoned on its very first launch?

It does not, and two independent mechanisms stop it.

- **The report is never sent.** `submit()` needs a pass before it will post anything — an Intent extra, the host's shelf, or its own mirror, and if all three are empty it returns without a request. A television that has never seen a host has none of the three, so the endpoint is never called and the row stays `unknown` forever.
- **And if it somehow were sent, the write would be refused.** `abandoned` requires `host_seen_at IS NOT NULL`. No host history, no mark.

So the only devices that can ever be marked are the ones that genuinely had a host and genuinely lost it. That is the line the product rule draws, enforced in two places rather than trusted in one.

Part 10 Three rules that must not be broken

1. It fails open

Only the exact string `inactive` locks. `active`, `unknown`, a value a newer server invented, a null body, a timeout, or no host installed at all — all leave the user alone. If you ever change this to lock on a broader set, you are choosing to lock people out when the network hiccups.

2. Only an explicit *false* counts

Both host signals are tri-state, and `null` is the default and the safe answer. It is omitted from the request body entirely and the server treats a missing field as no information at all. Null covers: no host installed, a host too old to publish the column, a provider that threw, a host that could not read its own store, and a provider running without a context. Only a host that is present and says plainly that it holds no session produces a `false`.

3. The machine mark must stay reversible

`abandoned` describes hardware and is inherited by any account that signs in on that hardware. A permanent version of that is a support nightmare and an unrecoverable customer. It clears on a strictly newer pass, it clears across the whole machine, and pairing is wired to force the mint that does it. All three of those are load-bearing; removing any one of them re-creates the lockout observed on 12 September 2026, where a re-paired television sat on `abandoned` with `host_seen_at` advancing and `host_minted_at` frozen.

Part 11 Operating it

Reading a single device row

```
SELECT id, user_id, device_name, entitlement_status,
host_seen_at, host_first_seen_at, host_minted_at, host_expires_at,
host_license, device_fingerprint
FROM bluehive.devices
WHERE user_id = :uid
ORDER BY host_seen_at DESC NULLS LAST;
```

The two timestamps to compare are `host_seen_at` and `host_minted_at`. **Seen advancing while minted stays frozen is the signature of a device that is reporting faithfully but carrying a pass nobody is refreshing** — which is what a re-pair looked like before pairing forced a mint. All timestamps on the server are UTC.

Symptom to cause

Symptom	Most likely cause	Check
Locked out right after re-pairing	The pass was not re-minted, so nothing is strictly newer than the abandonment.	`host_minted_at` older than the unpair. Press **Retry connection**, which forces a fresh read and wakes the host.
Nothing in the column ever changes, for anybody	The signing keypair has drifted apart, and every pass is being rejected with a 400.	Compare the 16-hex key fingerprints in both services' boot logs.
A whole account is locked and nobody knows why	Rule 3, the 7-day account-wide span rule, or rule 4.	Look for any row on that account with `entitlement_status = 'inactive'` and a `host_seen_at - host_first_seen_at` span of 7 days or more; then for any row on `blocked`.
A device locks on every launch for no apparent reason	The machine is marked, possibly under a different account.	Query every row sharing that `device_fingerprint`, not just the customer's own.
An operator block appears not to work	It is almost certainly working. The client only finds out on its next gated call.	Confirm the 403 in the API logs rather than watching the television.
Counts look frozen in the daily census	The background container is not running the loops.	`RUN_BACKGROUND_TASKS=1` on *bluehive-tasks*. The API container does not run them, and grepping its logs will tell you the loops are absent when they are merely elsewhere.

Where everything lives

File	Side	Responsibility
`auth/HostEntitlementRelay.kt`	BlueHive	The whole client side: reads both shelf columns, relays, records the verdict, owns the fail-open rule.
`auth/EntitlementLockBus.kt`	BlueHive	Plain listener registry carrying “standing became inactive” from a background coroutine to an application-level callback.
`auth/ServiceLockoutActivity.kt`	BlueHive	The deliberately vague connection screen, and the retry that wakes the host.
`entitlement/HostAssertion.kt`	OGD	Fetches and caches the pass. Owns the 6-hour gate and the forced re-mint.
`entitlement/HostAssertionProvider.kt`	OGD	Publishes the two-column shelf. Cache-only by design.
`host/HostUnpair.kt`	OGD	Server-side session end, local wipe, cache clear, and the kick to the pairing door from wherever the caller was.
`host/HostEventStream.kt`	OGD	The event stream, the hourly top-up tick, and the forced re-mint on a subscription change.
`routers/events.py`	platform-api	Mints and signs the pass. Holds `_is_entitled()`, the single definition of “may be used”.
`host_entitlement.py`	bluehive-api	Verifies the signature, derives the verdict, and owns every write guard in Part 7.
`bh_auth.py`	bluehive-api	The four refusal rules. Runs on every gated request.
`entitlement_health.py`	bluehive-api	The hourly expiry sweep and the daily census.

Part 12 Known gaps, in the interest of not lying to yourself later

- **Unpair is not pushed to BlueHive.** It is discovered on BlueHive's next launch or home-screen resume, not the instant it happens. Closing it means registering BlueHive as an Off-Grid Drive companion, which would give it an instant kick at the cost of BlueHive knowing Off-Grid Drive exists.
- **A host self-update is a narrow false-positive window.** Android reports a package as missing while it is being replaced, and the debounce that used to cover that is gone. A BlueHive launch timed inside that window marks the machine. It self-heals, because the returning host mints and the mark lifts — but the clean fix is for Off-Grid Drive to force a mint on start after an update, the same way pairing now does.
- **Two comments in the source are now stale.** `host_entitlement.py` still says “the client requires several readings spread over minutes before it will send False”, which stopped being true on 12 September 2026. `HostEventStream.kt` still refers to a “24h gate” where the gate has been 6 hours for some time. Neither changes behaviour; both will mislead somebody reading the code later.
- `/opt/bluehive` **is not a git repository.** Every change to these files is protected only by hand-made `.bak-` copies sitting next to them. That is a gap worth closing before any public release.
- **The** `blocked` **switch has no interface.** It is SQL typed by hand, which is fine while there is one operator and becomes a liability with two.